51 Additions per Recursive Step for Rank-23 3×3 Matrix Multiplication, Machine-Checked over Arbitrary Rings

Greg Sidebottom

October 11, 2026 — draft

Abstract

We present a rank-23 bilinear scheme for 3×3 matrix multiplication that uses 51 additions per recursive step in the alternative-basis (Karstadt–Schwartz) cost model, one below Nielsen's 52 (arXiv:2609.06588). The count 51 itself was first posted, on a different scheme, by Stapleton and Perminov (arXiv:2609.32388, September 26, 2026), found by heuristic sparse-basis search; the present work is independent and complementary: exactness and breadth rather than a single construction. The scheme is valid over every ring — the products keep their left/right factor order, so it recurses on block matrices — and its correctness is machine-checked in Lean 4/Mathlib down to the kernel (`Ring51.lean`; the core theorem depends on the axiom `propext` alone). The count is exactly optimal for the scheme's equivalence class: we introduce an exact free-basis minimization procedure whose negative answers are exhaustion proofs, and use it to show that 51 is the minimum for this class, that 52 is exactly optimal both for Nielsen's class and for the class of the current flat-model record, and to sweep the complete Heule–Kauers–Seidl database of rank-23 classes. Separately, we document a systematic gap phenomenon between $\mathbb{F}_2$ and signed optima: the $\mathbb{F}_2$-optimal chains of a class typically admit no sign assignment under most of the class's integer sign models, and certificate lifting fails where a direct signed search succeeds.

1 Introduction

A rank-r bilinear scheme for 3×3 matrix multiplication computes the 9 entries of $C = AB$ using r products of linear combinations of the entries of A and of B. Laderman's $r = 23$ scheme [1] remains the smallest known rank for the 3×3 format ($19 \le R(\langle 3,3,3 \rangle) \le 23$ over arbitrary fields [2]; over $\mathbb{F}_2$ the lower bound 20 was shown by Wang [3], and two September 2026 preprints claim 21 [4, 5]). With the rank fixed at 23, attention has turned to the *additive* cost of such schemes, in two distinct models.

The flat model. Count every binary addition or subtraction of a standalone straight-line program (negation free, no preprocessing). The published record chain runs Stapleton 60 [6] → Mårtensson–Wagner–Stapleton 59 [7] → Perminov 58 [8] → Sun 56 [9] → 55 (the author, Zenodo deposit of July 7, 2026 [10]; independently rediscovered by Karunaratne and Idamekorala on arXiv on July 28 [11] — same tensor, same 13+14+28 split, same transposed output construction).

The alternative-basis model. Karstadt and Schwartz [12] observed that a scheme may be conjugated by invertible linear *basis changes* on the two inputs and the output. Used recursively, the basis changes are applied once per matrix, blockwise, at total cost $O(n^2 \log n)$ — asymptotically free — while the additions *inside* the changed basis are paid at every recursive step and multiply

the leading constant of the $O(n^{\log_3 23})$ term. Their flagship result reduced Strassen's 2×2 step from 15 to 12 additions. For rank-23 3×3, sparse-decomposition work of Beniamini and Schwartz [13] gave 61 per step; Nielsen [14] gave 52, searching random basis changes over $\mathbb{F}_2$ and lifting to general fields; and Stapleton and Perminov [15] gave 51 on a flip-graph-derived scheme, with remarkably sparse conversions (five additions in total, so their complete one-shot algorithm costs 56 in ordinary coordinates). We prove below that their kernel, too, is exactly optimal for its class. The two models price different objects and are not comparable entry-by-entry: a flat scheme is an alternative-basis scheme with identity bases, so the alternative-basis optimum can only be smaller, and conversely a per-step count omits the (amortized) basis-change additions that the flat model charges.

Contributions.

1. **Two further 51-addition-per-step classes** (Section 4), valid over every ring: the Heule–Kauers–Seidl (HKS) classes `i12w211c23fg` and `i12w219c23fg`, found by exact search rather than sampling, and inequivalent to each other and to the scheme of Stapleton and Perminov [15] — so at least three distinct classes attain 51.

2. **Exact per-class optimality** (Section 3): an exact *free-basis minimization* procedure for one leg of a scheme, with machine-checkable certificates for upper bounds and finite exhaustions for lower bounds. The three legs decouple, so per-class per-step optima are exact: 51 is optimal for its class; 52 is exactly optimal for Nielsen's class (his published count, now with a matching lower bound) and for the class `i19w225c4efh` of the flat-model record (down from 55 flat).

3. **A database-wide sweep** (Section 5): a basis-invariant counting floor for every class of the HKS database of 17,376 rank-23 schemes, and an exact sweep of all classes whose floor permits ≤ 52. [Sweep in progress at the time of this draft; the table below reports its current state.]

4. **The $\mathbb{F}_2$–$\mathbb{Z}$ sign gap** (Section 6): the $\mathbb{F}_2$ optimum of a leg is often unattainable over $\mathbb{Z}$ under most of the class's sign models. For the 51-class, 20 of the first 22 sign models admit no signed chain matching the $\mathbb{F}_2$ counts, and certificate lifting (the standard route, used by Nielsen [14]) fails on 900 sampled certificates, 64 diagonal gauges and 128 models — while a direct signed search succeeds, instantly, at sign model 21.

5. **End-to-end machine verification** (Section 7): search certificates with independent replay; a word-ring verification of the assembled program over $\mathbb{Z}\langle A, B\rangle$; and a Lean 4 proof over arbitrary (non-commutative) rings, reproducible in seconds (Appendix A).

2 The cost model

A rank-23 scheme is a triple (U, V, W) of 23×9 integer matrices with $C[p,q] = \sum_{m=1}^{23} W_{m,(p,q)}\,(U_m\cdot \operatorname{vec} A)\,(V_m\cdot \operatorname{vec} B)$. Here vec flattens a 3×3 matrix row-major into a 9-vector (coordinate k is entry $(\lfloor k/3\rfloor,\ k \bmod 3)$). In the alternative-basis model one chooses invertible 9×9 matrices B_U, B_V, B_W (unimodular over $\mathbb{Z}$ for ring validity), computes $A' = B_U^{-1}\operatorname{vec} A$ and $B' = B_V^{-1}\operatorname{vec} B$ once per matrix, runs the recursion entirely in the primed coordinates, and decodes the output once. The *per-step cost* is the number of additions needed to evaluate the three in-basis linear maps: the two input legs UB_U, VB_V (each 23 forms from 9 inputs) and the output leg (the 9×23 map $m \mapsto (WB_W^{\top})^{\top} m$, priced through the transposition principle).

Proposition 2.1 (transposition overhead)**.** *For a* 9×23 *output map with no zero rows or columns, the minimal addition count equals that of its* 23×9 *transpose plus* $23 - 9 = 14$*.*

This is Tellegen's principle, used constructively throughout (the flat record's 28-addition output is a transposed 14-addition chain [10, 11]; the 51 scheme's 26-addition output is a transposed 12-addition chain). Hence a per-step total is $a_A + a_B + (a_C + 14)$ where a_A, a_B, a_C are the three 23×9 leg costs, and 51 corresponds to legs $(12, 13, 12)$.

Lemma 2.2 (leg decoupling over rings)**.** *Over any ring, with negation free, the per-step optimum of a class is the sum of the three leg optima, and the per-product sign constraint* $\varepsilon_u \varepsilon_v \varepsilon_w = +1$ *is vacuous: leg cost is invariant under per-row sign flips, and whatever signs the three chains realize are absorbed at the point of use (a* -1 *coefficient in the output assembly is free).*

Proof. A chain value used with a minus sign costs nothing extra in the negation-free model, so replacing any target row by its negation changes no leg cost. Given chains realizing $\varepsilon_u^{(m)} u_m$, $\varepsilon_v^{(m)} v_m$, $\varepsilon_w^{(m)} w_m$, use the output value of product m with coefficient $\varepsilon_u^{(m)} \varepsilon_v^{(m)} \varepsilon_w^{(m)}$; the composite computes $\sum_m (u_m A)(v_m B)\, w_m$ exactly. □

3 Exact free-basis minimization

Fix one leg: a 23×9 matrix X over $\mathbb{F}_2$ (or over $\mathbb{Z}$, with targets taken up to sign) with d distinct nonzero rows. The problem is to minimize, over invertible B, the number of additions needed to compute all rows of XB from the 9 transformed inputs. Equivalently: choose any 9 linearly independent *free* forms, then build every distinct target by pairwise additions.

Proposition 3.1 (structure of optimal chains)**.** *A chain with* a *additions,* k *of whose* 9 *free forms are targets, and* h *helper (non-target) additions, satisfies* $a = (d - k) + h$*. Consequently* $a = d - 9$ *iff some independent* 9*-target basis closes to all of* T *under the waste-free closure (repeatedly add any pair-sum that is an unbuilt target);* $a = d - 8$ *iff a* 9*-target basis plus one helper, injectable at the closure fixpoint, suffices, or an independent* 8*-target basis plus one arbitrary free vector; and so on.*

Two facts make this an exact procedure. The waste-free closure is *monotone* — available forms never hurt — so greedy closure decides waste-free reachability. And every form preceding the i-th helper lies in the pure closure of the basis and the earlier helpers, so helpers are enumerable at successive fixpoints: the case sweep is finite and complete for the 9-target-basis configurations. (The tiers with free non-target basis vectors are finite over $\mathbb{F}_2$ — 511 vectors — and were swept there; over $\mathbb{Z}$ they are unbounded and are the one stated scope caveat of the signed lower bounds; see Section 6.) Every positive answer emits the chain, which an independent checker replays (basis independence and, over $\mathbb{Z}$, unimodularity by exact determinant; each addition; coverage) before it is reported; every negative answer is a finite exhaustion.

Proposition 3.2 (basis-invariant floor)**.** *In every basis, an input leg needs at least* $d - 9$ *additions (*d *= distinct rows; invariant under invertible column action), and the output leg at least* $(d-9)+14$*. Hence the per-step total of a class is at least* $d_A + d_B + d_C - 13$*.*

4 The 51-addition scheme

Running the procedure on the class `i12w211c23fg` (HKS family `17x2+52x+4y6+5y4+14y3+z32+2z27`) over $\mathbb{F}_2$ gives exact leg minima

$$a_A = 12, \qquad a_B = 13, \qquad a_C = 12,$$

each with a certificate, and with a=11 exhausted on all three legs and a=12 exhausted on the B leg. By Lemma 2.2 the class's per-step optimum is exactly

$$12 + 13 + (12 + 14) \;=\; \mathbf{51}.$$

The ring-valid realization comes from the class's integer sign model 21 (Section 6): signed chains with the same counts, from unimodular signed bases, assembled into the scheme that Appendix A exhibits in full as a Lean definition, together with the kernel-checked theorem that it computes AB over every ring.

The conversions of this realization cost $4 + 7 + 3 = 14$ additions — minimized *exactly*, by sweeping every feasible basis at the optimal leg counts and scoring $\sum(\text{weight} - 1)$ over the nine basis rows; the unconstrained envelope from the class's row weights is 11. The complete one-shot algorithm therefore costs $51 + 14 = 65$ additions in ordinary coordinates; for comparison, Stapleton and Perminov's conversion-optimized class achieves $51 + 5 = 56$ [15], and the flat-model record is 55 [10, 11]. At the optimal counts the accounting $a = (d - k) + h$ forces at least eight basis rows to be target rows, so conversion cost is governed by the class's light-row supply rather than by free design — near-identity conversions like [15]'s exist exactly when the class has many weight-one rows.

The second class, `i12w219c23fg`, has exact legs $(11, 13, 13)$: its A leg *achieves the counting floor* $d - 9$ — a 9-target unimodular basis whose waste-free closure covers all 20 targets, the only floor-achieving leg observed in the database so far.

For calibration, the same exact procedure gives: for Nielsen's base class `i12w195c23ci`, legs $(12, 12, 14)$ — so his published 52 [14] is *exactly optimal* for his class, a lower bound his sampling-based search could not provide; for the scheme of Stapleton and Perminov [15], legs $(13, 12, 12)$ — their heuristically-found 51 kernel is likewise exactly optimal for its class, which is inequivalent to both of ours; and for `i19w225c4efh` (the class of the flat-model record), legs $(13, 13, 12)$, i.e. per-step 52, down from 55 flat — with the curious feature that its *output* leg, at 14 flat, is the leg that drops to 12 under basis change.

5 Database sweep

Computing (d_A, d_B, d_C) for all 17,376 schemes of the HKS database takes seconds and yields the floor distribution of Proposition 3.2: minimum 47, attained by 44 schemes (including the three classes above); 507 schemes at 48; Laderman alone at 56 ($d = (23, 23, 23)$: no shared factors, maximally unsuited to the model). The exact sweep of every class with floor ≤ 48 (303 classes after deduplication) is running at the time of this draft; its state as of October 12 (186 of 303 classes decided):

per-step total	classes	note
51	2	`i12w211c23fg`, `i12w219c23fg` (exact; §4)
52 (tie)	13	eleven in the `i12` family
53	17	incl. Stapleton's class at $(13, 13, 13)$
54	14	
55	3	
ruled out for ≤ 51	136	leg-sum lower bounds ≥ 38

[Final table to follow; the sweep is exact on each decided class.] We also profiled and swept our own corpus of 123 rank-23 schemes outside the sweep above: 53 walk-discovered classes certified

inequivalent to the entire database, plus 70 later finds (not yet equivalence-checked against it). Of these, 16 were eligible at floor ≤ 48, and all 16 are ruled out for ≤ 51 — the database's optimum stands across every class known to us. A 50 would require exact leg sum 36. The closest approach so far is slack +2 over the leg floors (the $(11, 13, 13)$ class); exact 11-legs exist, so sum 36 is not excluded by any per-leg phenomenon observed to date. Because leg minima only rise with d and floor-49 classes would need slack +1 — never observed — the floor-≤ 48 sweep settles the ≤ 51 question for the entire database.

6 The sign gap

An $\mathbb{F}_2$ result lifts to the ring-valid statement only if the chains admit consistent signs. A mask scheme has many integer *sign models* — solutions of the signed Brent equations, which we enumerate by SAT with blocking clauses — and the lift target is model-dependent. Empirically the dependence is drastic:

- **Certificate lifting fails systematically.** For `i19w225c4efh`'s B leg, none of 301 $\mathbb{F}_2$-optimal certificates admits any sign assignment under the database representative's model (256 basis signings each, all unimodular, all DFS-obstructed), nor under all 64 diagonal sign gauges, nor under 128 SAT-enumerated models. The same held for the 51-class's A and B legs.
- **Direct signed search succeeds.** A signed variant of the free-basis procedure (targets up to sign; both signs tried per addition; $|\det|$ checked once per basis, as row signs do not change it) decides each model exactly. For the 51-class, models 0–20 fail (18 cannot reach $a_A = 12$; two reach it and exhaust $a_B = 13$) — and model 21 clears all three legs in seconds, B included.
- **The gap is real but model-local.** For `i19w225c4efh`, signed $a_A = 13$ and $a_C = 12$ hold with certificates, but $a_B \geq 14$ over all 9-target-basis configurations: the class's ring-valid per-step value is 52 or 53, with 52 confined to the unbounded free-vector-basis configurations (an SMT attack on a bounded version is in progress). Certificate lifting is thus doubly incomplete: it samples chains, and it misses signed chains whose mod-2 shadow is degenerate. Only the direct signed search decides.

7 Verification

Four independent layers check the 51 claim. (i) Every search certificate is replayed by a checker sharing no code with the search (chain validity, coverage, basis independence/unimodularity). (ii) The sign model is verified against the signed Brent equations. (iii) The generator that assembles the full scheme re-verifies the *emitted* program end-to-end over the word ring $\mathbb{Z}\langle A, B\rangle$ — all nine outputs equal AB as non-commutative polynomials, with the additions counted ($12 + 13 + 26 = 51$); this gate caught a genuine sign-bookkeeping bug on its first run. (iv) The Lean theorem of Appendix A is checked by the Lean 4 kernel against Mathlib's matrix product; the hypothesis-form theorem depends on `propext` alone, the Matrix-API form on Lean's three standard axioms, and the whole file re-elaborates in under ten seconds (the appendix header gives the exact command).

Artifacts. All artifacts are in the repository https://github.com/gsidebottom/logic: the Lean proof `proof/Mm/Ring51.lean`; the leg certificates, sign model and signed chains in `matmul/external/ks51/`; the search tools (`src/bin/fbmin.rs`, `src/bin/fbsmin.rs`) and drivers (`matmul/fbsweep.py`, `matmul/fbsloop.py` and the generator `matmul/gen_ring51_lean.py`, whose word-ring gate re-verifies the emitted

program on every run. An archival deposit of these artifacts (paper draft, standalone proof, Lake project, certificates, tools, and a repository snapshot at tag `v1.0-ks51`) is published at doi:10.5281/zenodo.23305912.

8 Open problems

(1) Is the ring-valid optimum of `i19w225c4efh` 52 or 53? Equivalently: can a free-vector basis beat every 9-target basis over $\mathbb{Z}$? (2) Does any rank-23 class reach per-step 50, i.e. exact legs $(12, 12, 12)$? The sweep will settle this for the known database; a structural lower bound better than Proposition 3.2 would settle it in general. (3) Characterize the sign models under which a class's signed leg minima match its $\mathbb{F}_2$ minima — model 21's success after twenty failures invites explanation. (4) The same machinery applies verbatim to 4×4 rank-47/48/49 schemes.

References

[1] J. Laderman, *A noncommutative algorithm for multiplying* 3×3 *matrices using* 23 *multiplications*, Bull. AMS 82 (1976) 126–128.

[2] M. Bläser, *On the complexity of the multiplication of matrices of small formats*, J. Complexity 19 (2003) 43–60.

[3] C. Wang, *A lower bound of 20 for the rank of* 3×3 *matrix multiplication over* $\mathbb{F}_2$, arXiv:2603.07280.

[4] C. Wang, *A lower bound of 21 for* 3×3 *matrix multiplication over* $\mathbb{F}_2$, arXiv:2609.06725.

[5] S. Yang et al., *A structural proof of the lower bound 21 for* 3×3 *matrix multiplication over* $\mathbb{F}_2$, arXiv:2609.18722.

[6] D. Stapleton, *A 60-addition, rank-23 scheme for exact* 3×3 *matrix multiplication*, arXiv:2508.03857.

[7] Mårtensson, Wagner, Stapleton, *A rank 23 algorithm for multiplying* 3×3 *matrices with an arithmetic complexity of 59*, arXiv:2601.05272.

[8] Perminov, *A 58-addition, rank-23 scheme for general* 3×3 *matrix multiplication*, arXiv:2512.21980.

[9] Y. Sun, *An exact 56-addition, rank-23 scheme for general* 3×3 *matrix multiplication*, arXiv:2604.27645.

[10] G. Sidebottom, *A 55-addition rank-23 scheme for* 3×3 *matrix multiplication — artifacts, verifier, and machine-checked proof*, Zenodo, July 7, 2026, doi:10.5281/zenodo.21240904.

[11] S. Karunaratne, A. Idamekorala, *55 additions suffice for* 3×3 *matrix multiplication at rank 23*, arXiv:2607.28676.

[12] E. Karstadt, O. Schwartz, *Matrix multiplication, a little faster*, J. ACM 67 (2020); SPAA 2017.

[13] G. Beniamini, O. Schwartz, *Faster matrix multiplication via sparse decomposition*, SPAA 2019.

[14] H. M. Nielsen, *A 52-addition, rank-23 scheme for exact* 3×3 *matrix multiplication*, arXiv:2609.06588.

[15] J. Stapleton, A. I. Perminov, *A 51-addition alternative-basis kernel for rank-23* 3×3 *matrix multiplication*, arXiv:2609.32388.

A The Lean definition and theorem

The following is the complete scheme definition and the matrix-product theorem, verbatim from `proof/Mm/Ring51.lean` (comments lightly ASCII-folded for typesetting; the repository file is authoritative). Sections marked `UNCOUNTED` are the amortized basis changes; the `COUNTED` sections contain the $12+13+26 = 51$ per-step additions. To re-check the proof from the repository (forcing re-elaboration rather than a cached replay):

```
cd proof && find .lake/build -name "Ring51.*" -delete && lake build Mm.Ring51
```

Expected: `Built Mm.Ring51` in 6–8 seconds against the prebuilt Mathlib cache, with axiom audits `[propext]` (for the hypothesis-form theorem `Ring51.correct`, not reproduced here) and `[propext, Classical.choice, Quot.sound]` for the theorem below. The ambient context is `variable {R : Type _} [Ring R]`.

```
/-- The 51-addition-per-step scheme as a map on `3x3` matrices, in the
structure of `Matmul55.scheme`: the two UNCOUNTED unimodular input
basis changes, the COUNTED 12- and 13-addition input chains, the 23
products, the COUNTED 26-addition transposed output stage, and the
UNCOUNTED output basis change.  Counted additions per recursive step:
12 + 13 + 26 = 51. -/
def rank23_add51_scheme (A B : Matrix (Fin 3) (Fin 3) R) :
    Matrix (Fin 3) (Fin 3) R :=
  -- A-side basis change (UNCOUNTED, amortized): the 9 transformed
  -- inputs, an invertible integer map applied once per matrix
  let ab0 := A 0 0
  let ab1 := A 1 0
  let ab2 := A 0 0 + A 2 0
  let ab3 := A 0 1 + A 2 1
  let ab4 := A 1 1
  let ab5 := A 2 1
  let ab6 := A 1 1 + A 1 2 - (A 2 2)
  let ab7 := A 2 2
  let ab8 := A 0 2
  -- A-side chain: 12 COUNTED additions (paid per recursive step)
  let aw0 := ab0 + ab5
  let aw1 := ab1 - ab7
  let aw2 := ab2 - ab7
  let aw3 := ab3 + ab8
  let aw4 := ab4 - ab8
  let aw5 := ab6 - aw1
  let aw6 := aw0 - aw3
  let aw7 := aw2 - aw3
  let aw8 := aw4 - aw5
  let aw9 := aw4 - aw6
  let aw10 := ab1 - ab2
  let aw11 := aw0 + aw10
  -- B-side basis change (UNCOUNTED)
  let bb0 := B 0 0
```

```
let bb1 := B 0 1 + B 1 1
let bb2 := B 0 2 + B 1 2
let bb3 := B 1 0 - (B 1 1) - (B 2 0) + B 2 1
let bb4 := B 1 1
let bb5 := B 1 2
let bb6 := B 2 0 - (B 2 2)
let bb7 := B 1 1 - (B 2 1)
let bb8 := B 2 2
-- B-side chain: 13 COUNTED additions
let bw0 := bb0 - bb2
let bw1 := bb0 + bb8
let bw2 := bb5 + bw0
let bw3 := bb6 + bw1
let bw4 := bb7 + bb8
let bw5 := bb5 - bw4
let bw6 := bb6 + bw4
let bw7 := bb4 - bw6
let bw8 := bw1 - bw2
let bw9 := bb0 - bb1
let bw10 := bw6 + bw9
let bw11 := bb3 + bw0
let bw12 := bw6 + bw11
-- the 23 products (A factor on the left: recurses on blocks)
let m1 := aw0 * bw0
let m2 := aw1 * bw1
let m3 := aw2 * bb0
let m4 := ab0 * bb1
let m5 := ab1 * bw10
let m6 := ab2 * bw10
let m7 := ab0 * bb2
let m8 := ab1 * bw8
let m9 := aw11 * bw2
let m10 := ab3 * bb3
let m11 := ab4 * bb3
let m12 := ab5 * bw12
let m13 := aw6 * bb4
let m14 := aw4 * bw4
let m15 := aw3 * bw6
let m16 := aw9 * bb5
let m17 := ab4 * bw5
let m18 := ab6 * bb6
let m19 := ab7 * bw3
let m20 := ab8 * bb7
let m21 := aw5 * bw7
let m22 := aw7 * bw7
let m23 := aw8 * bb8
-- output stage: 26 COUNTED additions -- the Tellegen transpose
-- of the certified 12-addition C chain (12 + (23-9) fan-in = 26)
let z0 := m3 + m19
let z1 := m14 + m20
let z2 := z1 + m17
let z3 := m23 - z2
let z4 := m2 + m19
let z5 := z4 - z3
let z6 := m1 - z0
let z7 := z6 + m7
let z8 := m15 - m20
let z9 := z8 + z0
let z10 := m18 - m17
```

```
  let z11 := z10 + z5
  let z12 := m4 - m13
  let z13 := z12 - m20
  let z14 := m5 - m21
  let z15 := z14 - z11
  let z16 := m6 + m22
  let z17 := z16 - z9
  let z18 := m8 - z3
  let z19 := m9 - z5
  let z20 := z19 - z7
  let z21 := m10 + z9
  let z22 := m11 + z11
  let z23 := m12 - z7
  let z24 := m16 - z2
  let z25 := z24 + m7
  -- output basis change (UNCOUNTED): the inverse transform,
  -- rows of the unimodular output basis
  let c11 := z21 - z23
  let c12 := z13
  let c13 := z25
  let c21 := z22
  let c22 := -z15
  let c23 := z18
  let c31 := z23
  let c32 := -z13 - z17
  let c33 := z18 + z20
  !![c11, c12, c13;
     c21, c22, c23;
     c31, c32, c33]

-- `rank23_add51_scheme` unfolds to a ~120-binding straight-line
-- program; a single elaboration exceeds the default heartbeat budget.
set_option maxHeartbeats 1600000 in
-- Plain `simp` evaluates the concrete `!![..] i j` indexing via
-- Mathlib's matrix simprocs; the explicit lemma set is version-fragile,
-- so the flexible-`simp` lint is accepted here (as in Mm/Matrix.lean).
set_option linter.flexible false in
/-- The scheme computes the matrix product, in Mathlib's own terms. -/
theorem rank23_add51_scheme_eq_mul (A B : Matrix (Fin 3) (Fin 3) R) :
    rank23_add51_scheme A B = A * B := by
  simp only [rank23_add51_scheme]
  ext i j
  fin_cases i <;> fin_cases j <;>
    simp [Matrix.mul_apply, Fin.sum_univ_three] <;>
    noncomm_ring
```